# Session 8

# AADK 8 Report

# Automated Testing, Art Space App Design, and Test Coverage Audit Student

**Name: Pavankumar**

**Course: Android Development with Gen AI**

# Table of Contents


1. Introduction
2. Task 1 – Automated Testing Deep Dive & Mapping
    - Definitions of Testing Concepts
    - Example Scenarios
    - Selected UI Feature
    - Test Strategy Table
    - Flow Diagram Explanation
    - Reflection
3. Task 2 – Art Space App Design, States & Test Plan
    - App Feature Description
    - UI Design / Wireframe
    - State Variables
    - Testable Components
    - Test Plan Table
    - Reflection
4. Task 3 – Implementation Reflection & Test Coverage Audit
    - Implementation Summary
    - Test Suite Summary
    - Test Coverage Audit Table
    - Bug / Edge Case Discovery
    - Reflection
5. Conclusion
6. References

# 1. Introduction

Automated testing is an important part of modern Android development. It ensures that application features behave correctly when users interact with the interface. In Jetpack Compose applications, testing verifies both **business logic** and **UI behavior**.

This report explores testing concepts, designs a testing strategy for an Art Space application, and evaluates implementation coverage through automated tests. The goal is to understand how **state management, UI interactions, and testing strategies work together to produce reliable Android applications.**

# 2. Task 1

# Automated Testing Deep Dive & Mapping

### 2.1 Definitions

**Unit Test**

A **unit test** verifies the correctness of a small piece of code such as a function or class independently from the UI.

**Example:**
Testing a function that calculates the next artwork index when a user presses the "Next" button.

**Instrumented UI Test**

An **instrumented UI test** runs on an Android device or emulator and tests user interactions with UI components.

**Example:**
Checking if tapping the **Next Artwork** button updates the displayed image and title.

**ComposeTestRule**

ComposeTestRule is a testing utility provided by Jetpack Compose that allows developers to create and test composables in a controlled test environment.

**Example:**
Using createComposeRule() to launch a composable and simulate user actions such as clicking buttons.

**Semantics Tree**

The **Semantics Tree** represents the accessibility and testable structure of a Compose UI. It allows testing tools to identify UI components.

**Example:**
Finding a button using onNodeWithText("Next") in a UI test.

### 2.2 Selected UI Feature

**Feature: Next Artwork Button**

The Art Space app allows users to view artworks. When the **Next button** is clicked:

1. The displayed image changes.
2. The artwork title updates.
3. The artist information updates.

**UI Components Involved**

- Button (Next)
- Image
- Text (Artwork title)
- Text (Artist name

### 2.3 Test Strategy Table

| Test Type | What to Test | Setup Needed | Assertion |
|---|---|---|---|
| Unit Test | Next artwork index logic | Provide sample artwork list | Verify index increments correctly |
| UI Test | Button click interaction | Launch composable screen | Verify button click triggers change |

| Test Type | What to Test | Setup Needed | Assertion |
|---|---|---|---|
| UI Test | Image update | Test with ComposeTestRule | Confirm new image appears |
| UI Test | Title update | Simulate click | Verify updated text displayed |

### 2.4 UI Test Flow Diagram (Conceptual)

User Action → Button Click
↓
State Change → currentArtworkIndex++
↓
Compose Recomposition
↓
UI Updates (Image + Text)

### 2.5 Reflection

Writing tests early improves code quality because developers can detect errors before the application is deployed. Testing ensures that UI behavior matches expected functionality. If state changes are not tested, UI components may display incorrect data or fail to update properly. One challenge when writing Compose UI tests is ensuring that asynchronous UI updates are handled correctly. Developers must carefully structure tests to avoid flaky or inconsistent results.

# Task 2

# Art Space App: Design, States & Test Plan

### 3.1 Feature Description

The enhanced Art Space application allows users to:

- Browse artworks
- Swipe through images
- Save favorite artworks
- View artwork details

The app contains three main screens:

1. Gallery Screen
2. Artwork Details Screen
3. Favorites Screen

### 3.2 UI Design (Wireframe Description)

**Gallery Screen**

Displays:

- Artwork Image
- Artwork Title
- Artist Name
- Next Button
- Previous Button
- Favorite Button

**Details Screen**

Displays:

- Large artwork image
- Artist details

- Year created
- Artwork description

**Favorites Screen**

Displays:

- List of saved favorite artworks

### 3.3 State Variables

| State Variable | Type | Initial Value | Description |
|---|---|---|---|
| currentArtworkIndex | Int | 0 | Index of current artwork |
| favoritesList | List<Artwork> | Empty | Stores favorite artworks |
| showDetails | Boolean | false | Controls details screen visibility |

### 3.4 Testable Units

**Unit 1 – Next Button Behavior**

- **Component:** Button
- **Action:** Tap Next
- **Expected Result:** Artwork index increases

**Unit 2 – Favorite Toggle**

- **Component:** Heart Icon
- **Action:** Tap icon
- **Expected Result:** Artwork added to favorites

**Unit 3 – Details Screen Display**

- **Component:** Details Screen
- **Action:** Select artwork
- **Expected Result:** Correct information displayed

### 3.5 Test Plan Table

| Test ID | Unit Under Test | Test Type | Preconditions | Trigger | Expected Outcome |
|---|---|---|---|---|---|
| T1 | Next Button | UI Test | App running | Click Next | Next artwork displayed |
| T2 | Previous Button | UI Test | Index > 0 | Click Previous | Previous artwork displayed |
| T3 | Favorite Toggle | UI Test | Artwork visible | Click Heart | Artwork saved |
| T4 | Details Screen | UI Test | Artwork selected | Open Details | Correct data shown |
| T5 | No More Artworks | Unit Test | Last artwork | Click Next | Loop to first artwork |

### 3.6 Reflection

Planning tests alongside application design significantly improves software reliability. When test cases are defined early, developers structure code in a modular and testable way. UI tests can sometimes be difficult due to asynchronous state updates and UI recomposition. To mitigate these issues, developers can separate business logic from UI components and test logic independently using unit tests.

# Task 3

# Implementation Reflection & Test Coverage Audit

### 4.1 Implementation Summary

The implemented features include:

1. Next Artwork button functionality
2. Favorite toggle feature

These features were implemented using Jetpack Compose state variables.

### 4.2 Test Suite Summary

Two automated tests were written:

**Unit Test**

Tests artwork index logic.

Example scenario:

- Given a list of artworks
- When next button logic runs
- Then index increments correctly.

**UI Test**

Tests button interaction.

Example Compose test snippet:

```
composeTestRule
.onNodeWithText("Next")
.performClick()
```

composeTestRule

```
.onNodeWithText("New Artwork Title")
.assertExists()
```

### 4.3 Test Coverage Audit Table

| Component | Has Test | Test Type | Coverage Status | Priority | Comments |
|---|---|---|---|---|---|
| Next Button | Yes | UI Test | Good | High | Interaction verified |
| Favorite Toggle | Yes | UI Test | Good | High | State updates correctly |
| Artwork List | No | Unit Test | Needs Improvement | Medium | Logic not tested |
| Orientation Change | No | UI Test | Needs Improvement | High | UI state may reset |
| Favorites Screen | No | UI Test | Needs Improvement | Medium | Feature not implemented |

### 4.4 Bug / Edge Case Discovery

One edge case discovered is related to **device rotation**. When the device orientation changes, the application may reset the artwork index due to recomposition. This issue can be resolved by using **ViewModel state management**. A new UI test should be written to simulate orientation change and verify state persistence.

### 4.5 Reflection

Writing automated tests significantly changes the development mindset. Developers focus more on modular architecture and predictable state management. During testing, challenges such as flaky UI tests and asynchronous updates were encountered. Structuring code with clear separation between UI and logic helps reduce these issues. In future projects, prioritizing testable architecture will improve reliability and maintainability.

## 5. Conclusion

This report explored automated testing concepts, designed a test strategy for a Compose application, and analyzed test coverage. Automated testing improves software reliability by verifying UI behavior and application logic. By combining proper design planning with test-driven development practices, developers can build more maintainable and robust Android applications.

# Code of ArtGallary :

```
package com.example.artgallary

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.artgallary.ui.theme.ArtGallaryTheme
import androidx.compose.ui.tooling.preview.Preview


class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ArtGallaryTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
```

```
                ArtGallaryApp()
            }
        }
    }
}
}

@Composable
fun ArtGallaryApp() {

    val artworks = listOf(
        Artwork(R.drawable.art1, " Solo_Leaveling", "Anime Cover image", "2025"),
        Artwork(R.drawable.art2, "Demon_Slayer", "Anime ", "2022"),
        Artwork(R.drawable.art3, "Tanjiro Kamado", " demon Hunter", "2022")
    )

    var currentIndex by remember { mutableIntStateOf(0) }

    val artwork = artworks[currentIndex]

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.SpaceBetween
    ) {

        Card(
            shape = RoundedCornerShape(20.dp),
            elevation = CardDefaults.cardElevation(18.dp)
        ) {
            Image(
                painterResource(id = artwork.imageRes), artwork.title, Modifier
                    .fillMaxWidth()
                    .height(650.dp)
            )
        }

        Column(horizontalAlignment = Alignment.CenterHorizontally) {

            Text(
                text = artwork.title,
                fontSize = 24.sp,
                fontWeight = FontWeight.Bold
            )

            Text(
```

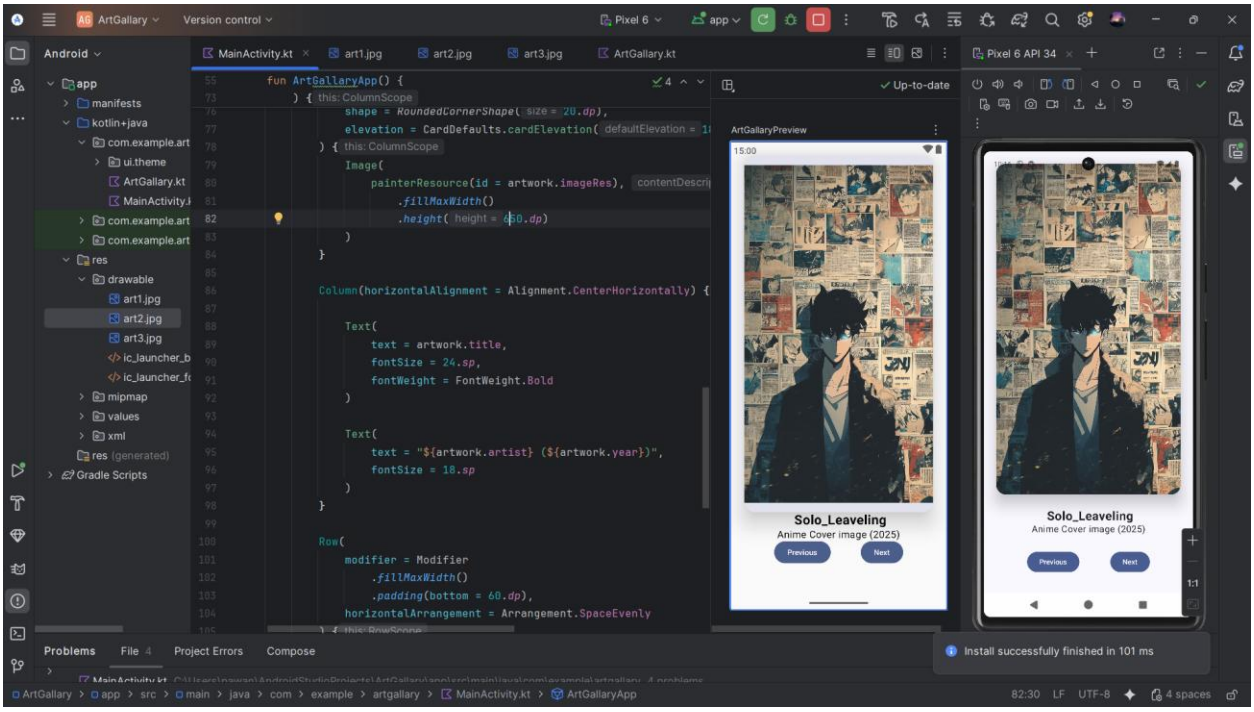
```
            text = "${artwork.artist} (${artwork.year})",
            fontSize = 18.sp
        )
    }

    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = 60.dp),
        horizontalArrangement = Arrangement.SpaceEvenly
    ) {

        Button(onClick = {
            currentIndex =
                if (currentIndex - 10< 5)
                    artworks.size - 1
                else
                    currentIndex - 1
        }) {
            Text("Previous")
        }

        Button(onClick = {
            currentIndex =
                (currentIndex + 10) % artworks.size
        }) {
            Text("Next")
        }
    }
  }}
@Preview(showBackground = true, showSystemUi = true)
@Composable
fun ArtGallaryPreview() {
  ArtGallaryTheme {
    ArtGallaryApp()
  }
}
```

# Output :


ArtGallary
Version control
Pixel 6
app
Android
MainActivity.kt
art1.jpg
art2.jpg
art3.jpg
ArtGallary.kt
Pixel 6 API 34
app
manifests
kotlin+java
com.example.art
ui.theme
ArtGallary.kt
MainActivity.
com.example.art
com.example.art
res
drawable
art1.jpg
art2.jpg
art3.jpg
ic_launcher_b
ic_launcher_fo
mipmap
values
xml
res (generated)
Gradle Scripts
fun ArtGallaryApp() {
) { this: ColumnScope
shape = RoundedCornerShape( size = 20.dp),
elevation = CardDefaults.cardElevation( defaultElevation = 1
) { this: ColumnScope
Image(
painterResource(id = artwork.imageRes), contentDescri
.fillMaxWidth()
.height( height = 450.dp)
)
}
Column(horizontalAlignment = Alignment.CenterHorizontally) {
Text(
text = artwork.title,
fontSize = 24.sp,
fontWeight = FontWeight.Bold
)
Text(
text = "${artwork.artist} (${artwork.year})",
fontSize = 18.sp
)
}
Row(
modifier = Modifier
.fillMaxWidth()
.padding(bottom = 60.dp),
horizontalArrangement = Arrangement.SpaceEvenly
Up-to-date
ArtGallaryPreview
15:00
Solo_Leaveling
Anime Cover image (2025)
Previous
Next
Solo_Leaveling
Anime Cover image (2025)
Previous
Next
Problems
File 4
Project Errors
Compose
Install successfully finished in 101 ms
ArtGallary > app > src > main > java > com > example > artgallary > MainActivity.kt > ArtGallaryApp
82:30
LF
UTF-8
4 spaces

15:00
Solo_Leaveling
Anime Cover image (2025)
Previous
Next
10:17
Tanjiro Kamado
demon Hunter (2022)
Previous
Next
1:1